



THE UNIVERSITY
WISCONSIN
MADISON

Accelerate Search and Recognition Workloads with SSE 4.2 String and Text Processing Instructions

Guangyu Shi, Min Li and Mikko Lipasti

University of Wisconsin-Madison

ISPASS 2011

April 12, 2011



Executive Summary

- STTNI can be used to implement a broad set of search and recognition applications
- Approaches for applications with different data structures and compare modes
- Benchmark applications and their speedup
- Minimizing the overhead



Introduction

- World data exceeds a billion billion byte ^[1]
- Search and Recognition applications are widely used
- Technical scaling: Improvement on clock frequency diminishing
- A novel way of implementing SR applications is needed.



Introduction

- SIMD: Single Instruction Multiple Data
- SSE: Streaming SIMD Extension (to x86 architecture)
 - Powerful in vector processing (graphics, multi-media, etc)
 - Limitations:
 - Larger register file consumes more power & area
 - Restriction on data alignment
 - Overhead on loading/storing XMM registers



Introduction: STTNI

- STTNI: String and Text processing Instructions
 - Subset of SSE 4.2, first implemented in Nehalem microarchitecture
 - Compare two 128-bit values in Bytes (8-bit * 16) or Words (16-bit * 8)
 - Format: opcode *string1*, *string2*, *MODE*



Introduction: STTNI

■ 4 STTNI instructions

Instruction	Description
pcmpestri	Packed compare explicit length strings, return index
pcmpestrm	Packed compare implicit length strings, return mask
pcmpistri	Packed compare explicit length strings, return index
pcmpestrm	Packed compare implicit length strings, return mask

		Source 2							
Source 1		a	t	a	d		t	s	T
	E	0	0	0	0	0	0	0	0
	x	0	0	0	0	0	0	0	0
	a	1	0	1	0	0	0	0	0
	m	0	0	0	0	0	0	0	0
	p	0	0	0	0	0	0	0	0
	l	0	0	0	0	0	0	0	0
	e	0	0	0	0	0	0	0	0
	lt	0	0	0	0	0	0	0	0
		1	0	1	0	0	0	0	0
		Result							



Introduction: STTNI

■ STTNI Mode options

Mode	Description
EQUAL_ANY	Element i in string 2 matches any element j in string 1
EQUAL_EACH	Element i in string 2 matches element i in string1
EQUAL_ORDERED	Element i and subsequent, consecutive valid elements in string2 match fully or partially with string1 starting from element 0
RANGES	Element i in string2 is within any range pairs specified in string1

Str 1	e	x	y	y	2	9	z	z
Str 2	e	x	a	m	p	l	e	x

1	1	0	0	0	0	1	1
---	---	---	---	---	---	---	---

1	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

1	1	0	1	1	1	1	1
---	---	---	---	---	---	---	---



Introduction

- STTNI invented for string, text and XML processing
- Operands not restricted to “strings and texts”
- Potential candidate for implementing Search and Recognition applications



Optimization with STTNI

- Classifying applications:

- By data structure

- Array
 - Tree
 - Hash Table

- By Compare mode

- Equality
 - Inequality

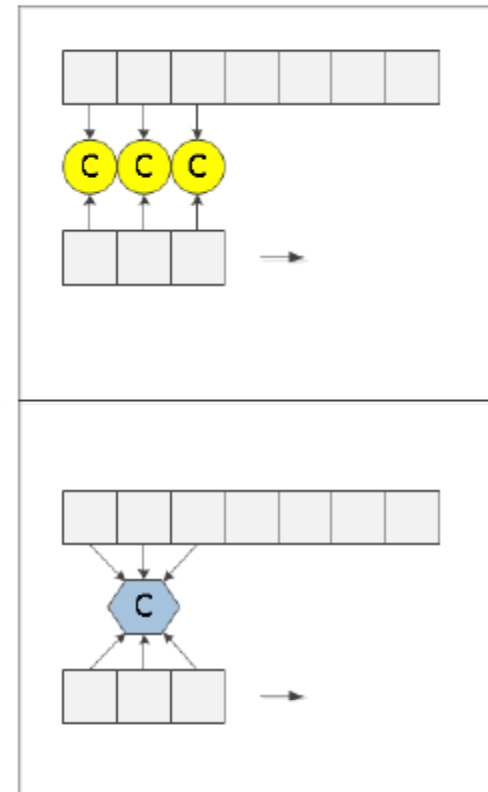


Optimization with STTNI

■ Optimization for different Data Structure

□ Array

- Linearly compare multiple elements in both arrays

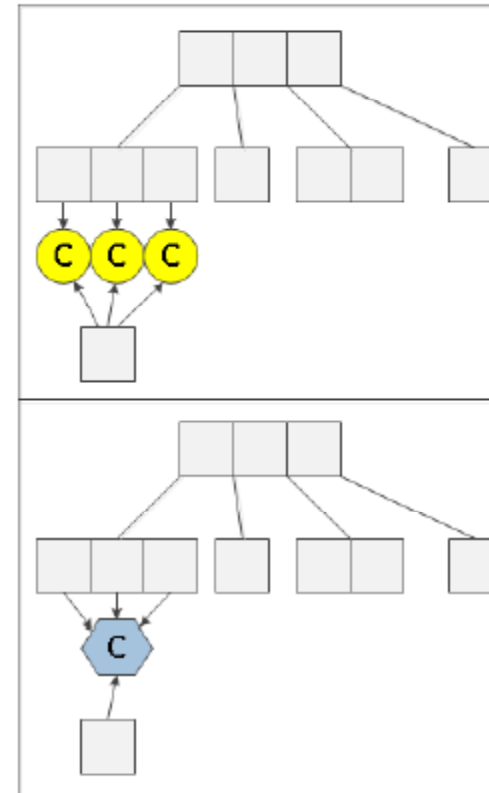




Optimization with STTNI

□ Tree

- Compare multiple words in a node

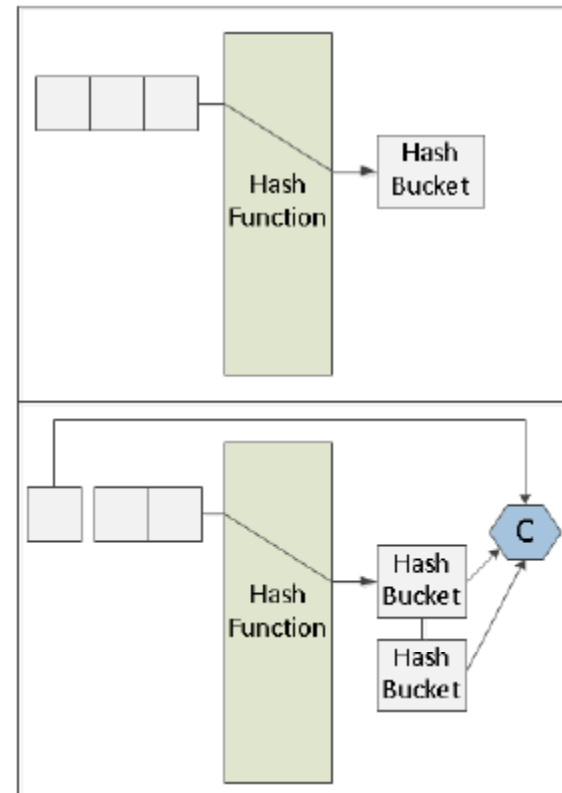




Optimization with STTNI

□ Hash Table

- Reduce number of entries by increasing hash collisions
- Resolving collisions is handled by STTNI
- Re-balance number of entries with maximum number of collisions





Optimization with STTNI

- Optimization for different Compare mode

- Equality

EQUAL_ANY, EQUAL_EACH, EQUAL_ORDERED

- Inequality

RANGES



Experimental Configurations

- Computer: Intel Core i7 (Nehalem) 2.8GHz
- L1 cache: 32KB, L2 cache: 256KB, both private
L3 cache: 8MB, shared
- Applications revised with STTNI manually
- Performance data are collected from built-in hardware counters
- Data normalized to baseline design (without STTNI-based optimization)



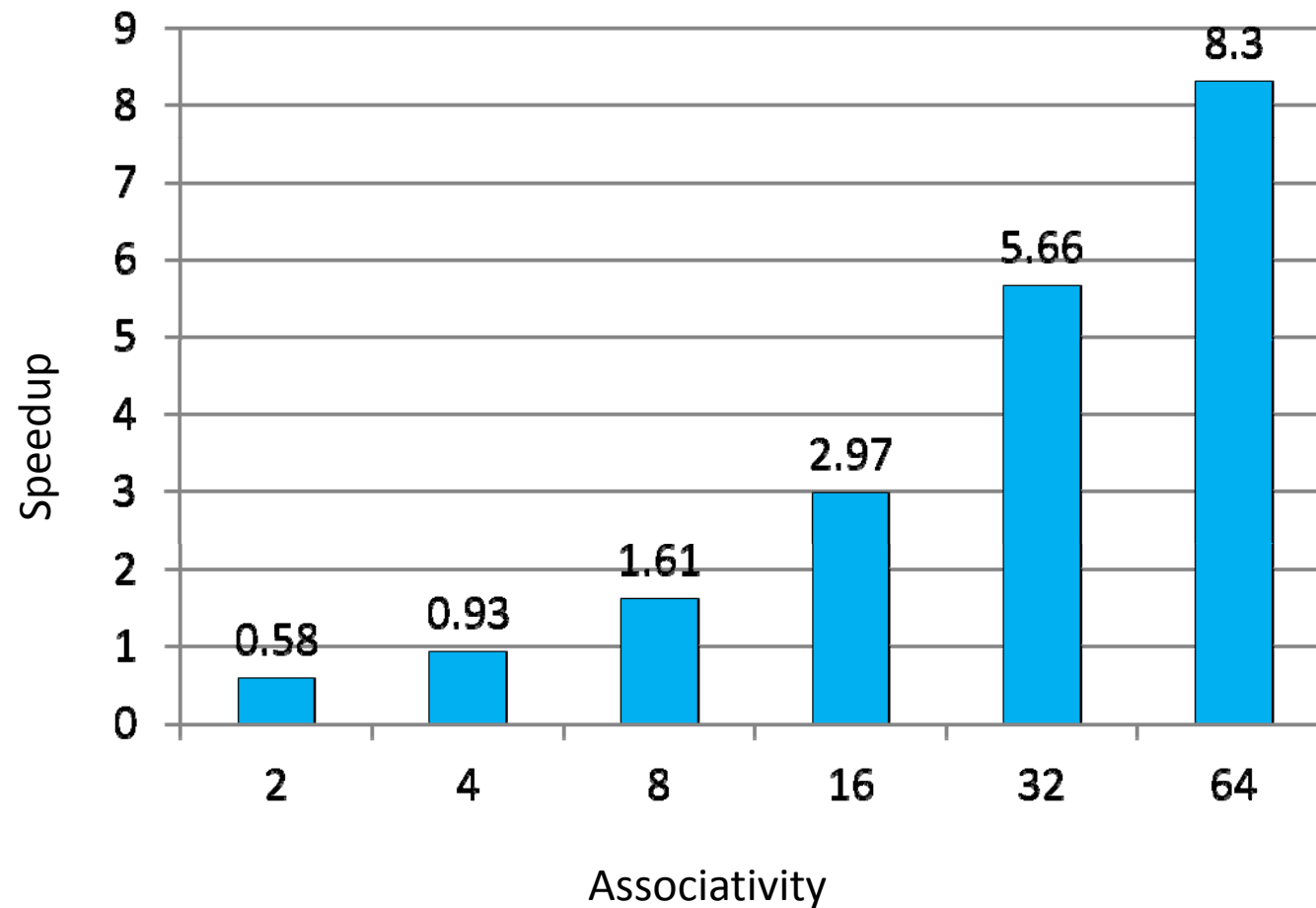
Benchmark Applications

Name	Field of Application	Data Structure	Compare Mode
Cache Simulator	Computer Simulator	Array	Equality
Template Matching	Image Processing	Array	Equality
B+Tree Algorithm	Database algorithm	Tree	Inequality
Basic Local Alignment Search Tool (BLAST)	Life Science	Hash Table	Equality



Experimental Results

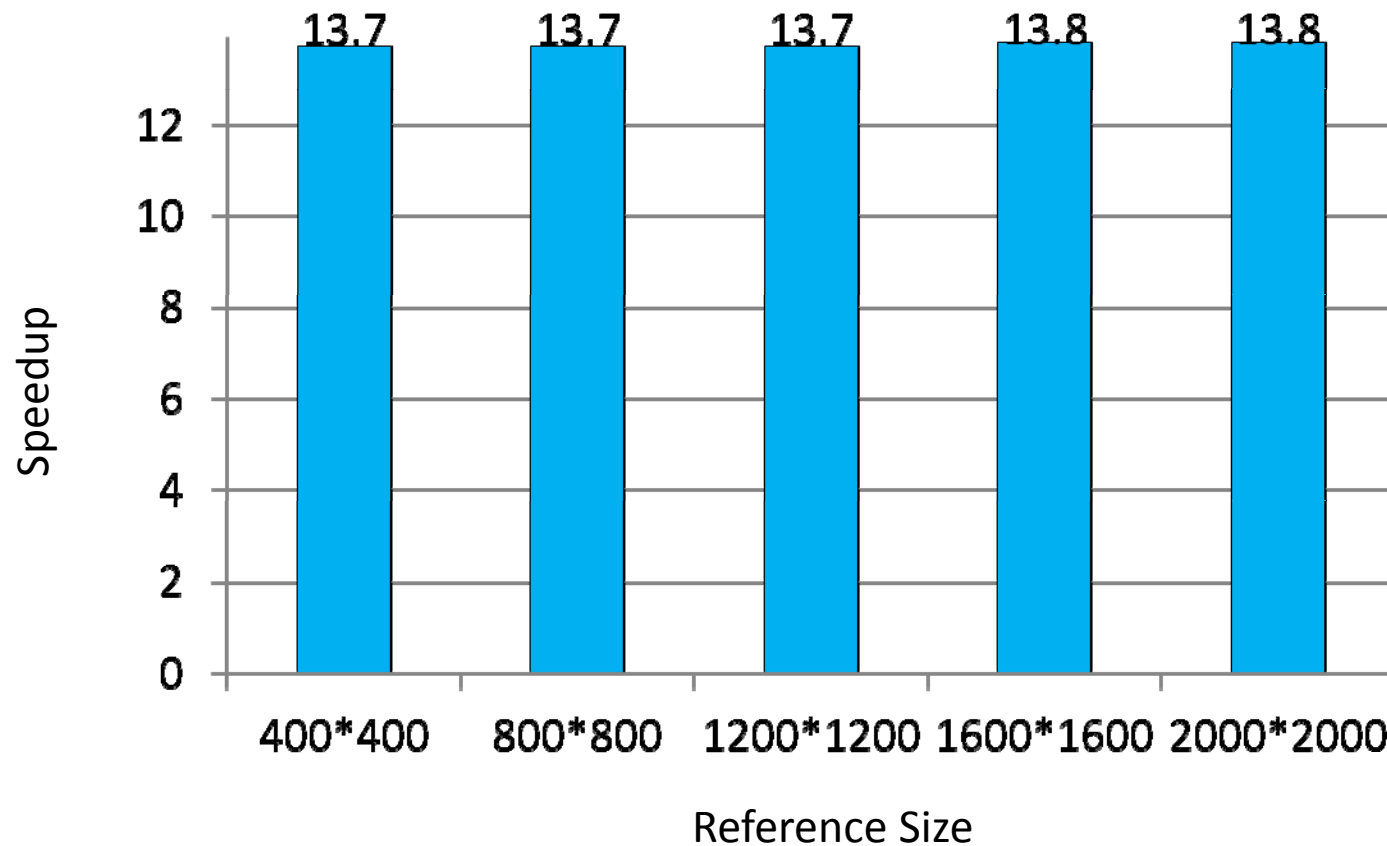
■ Cache Simulator





Experimental Results

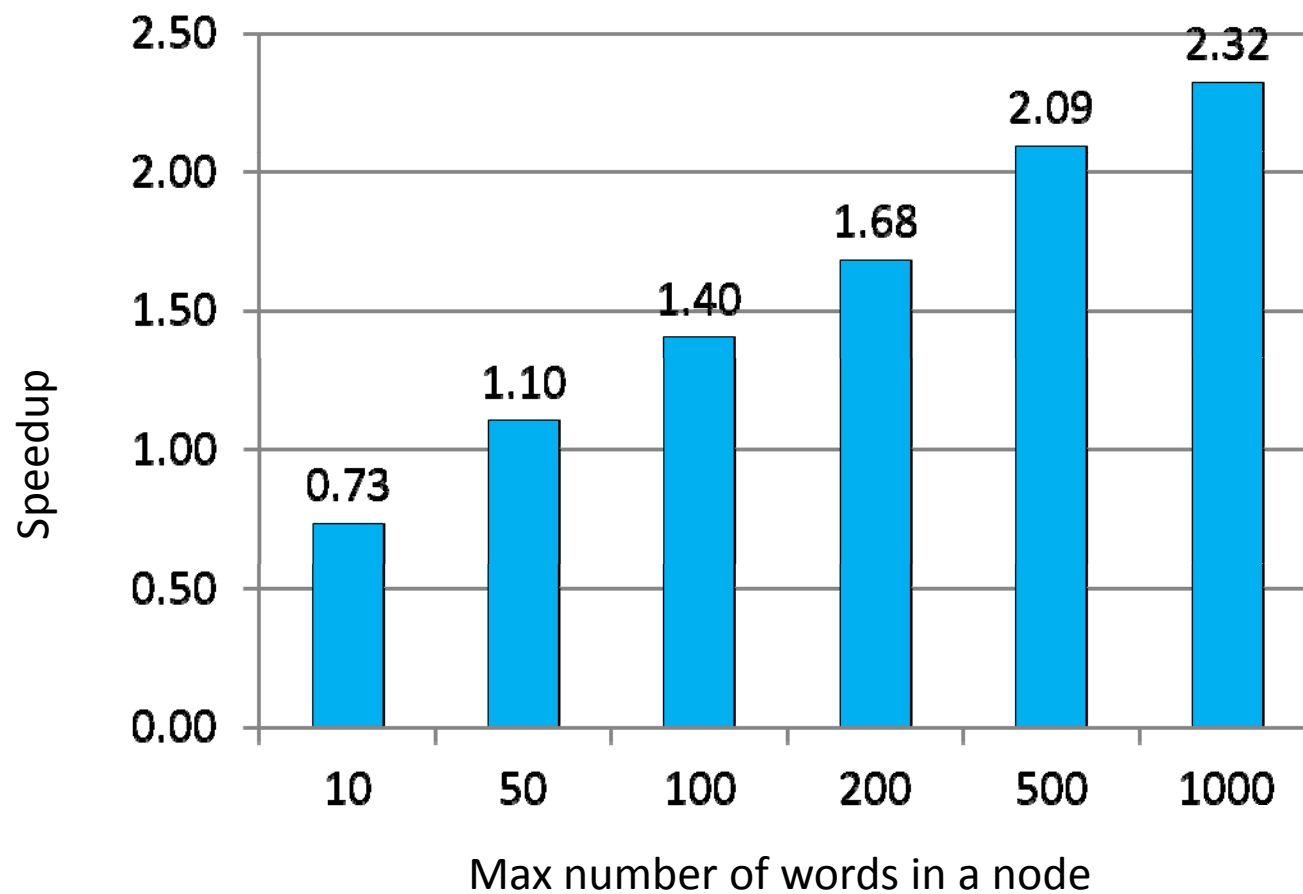
■ Template Matching





Experimental Results

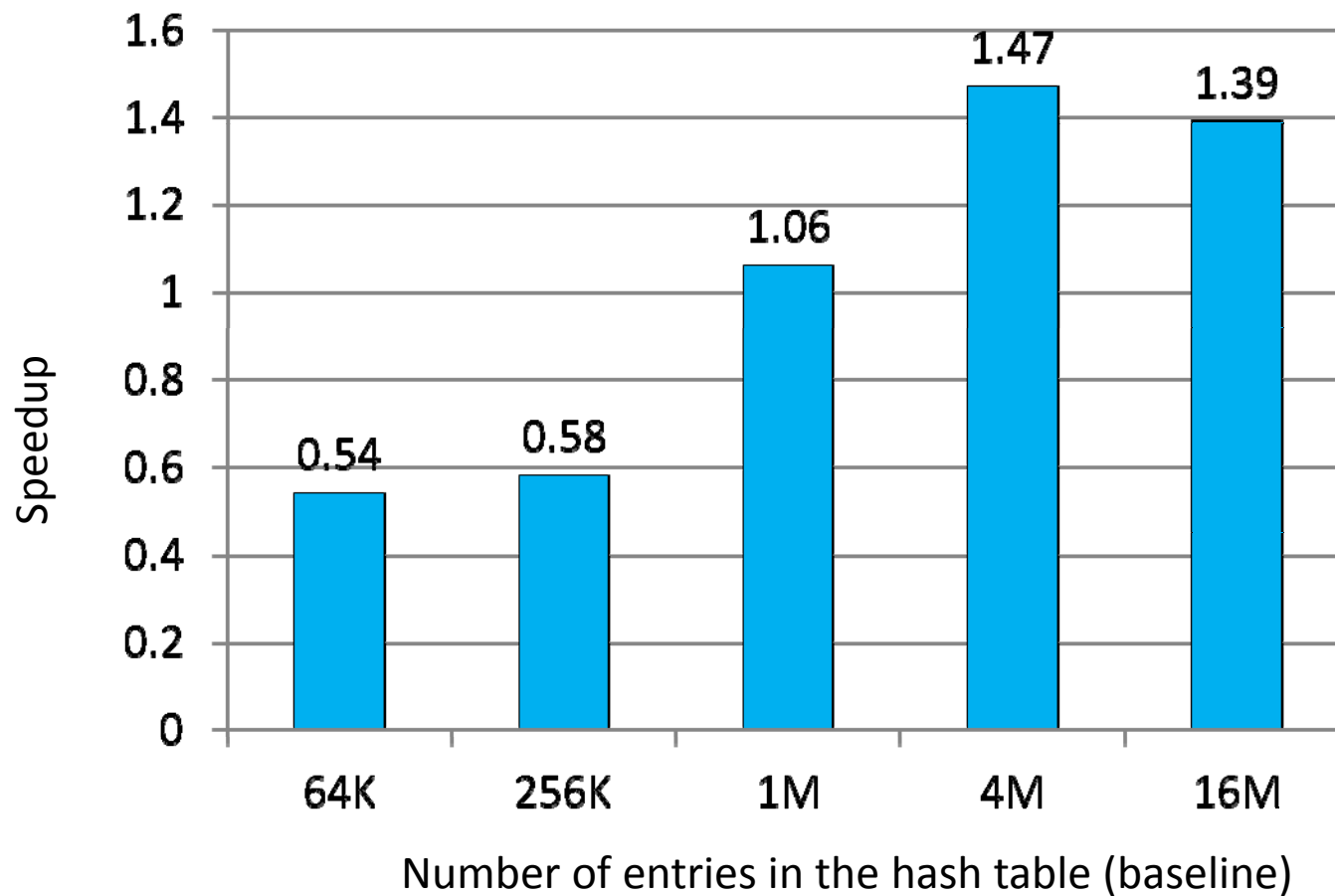
■ B+tree Algorithm





Experimental Results

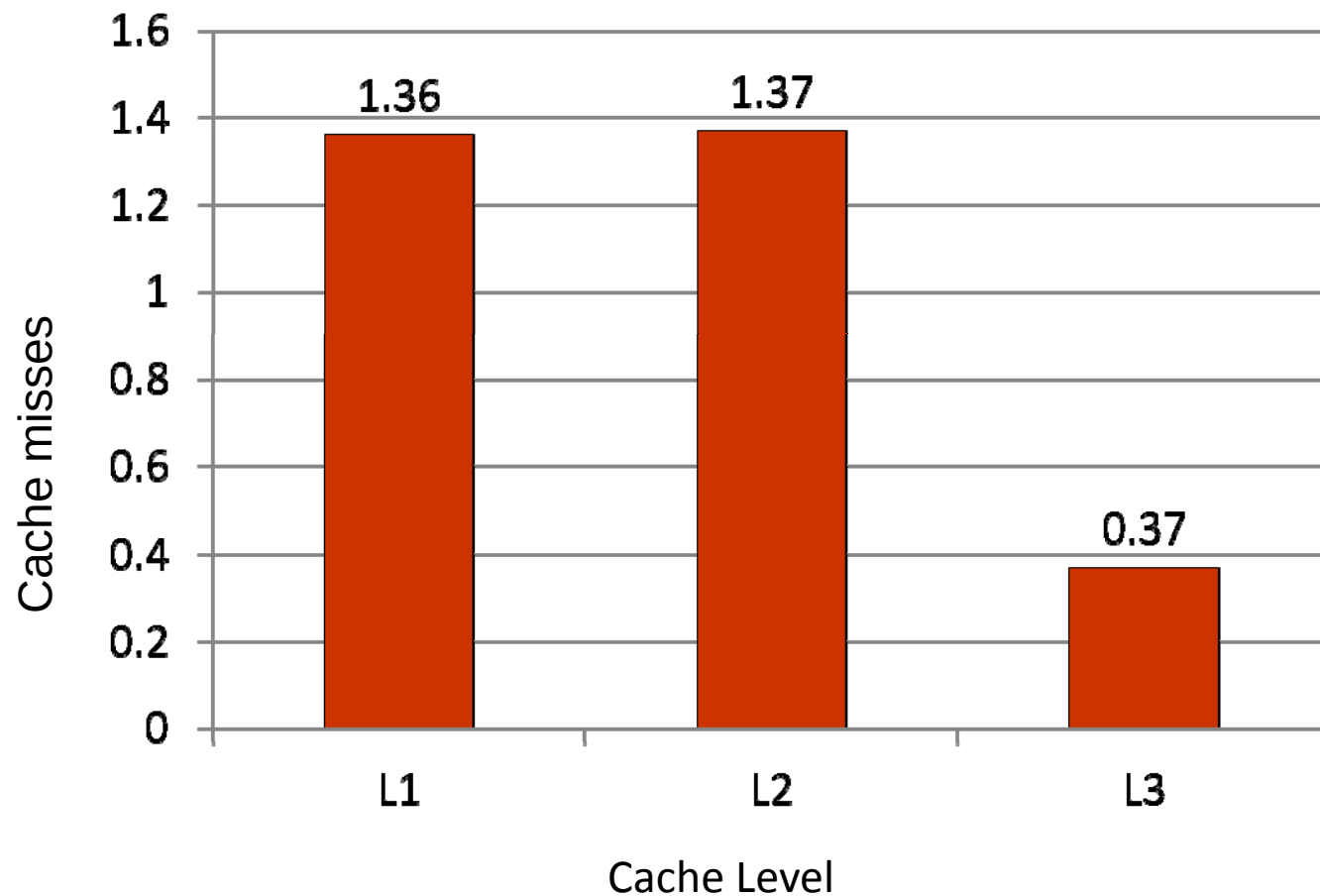
■ Basic Local Alignment Search Tool (BLAST)





Experimental Results

■ Basic Local Alignment Search Tool (BLAST)





Diverse Speedup

- Why the performance gains range from 1.47x to 13.8x for different benchmark applications?
 - Amdahl's Law
 - Different approaches for different data structure and compare mode
 - Overhead of STTNI



Minimizing the Overhead of STTNI

□ Source of overhead:

- 1. Initializing STTNI
- 2. Under-utilization of XMM registers
- 3. Loading/storing data from/to XMM registers

□ Solution:

- 1. Prefer longer arrays
- 2. Keep XMM register utilization high
- 3. Arrange data properly in memory



Conclusion and future work

- STTNI can be used to optimize a broad set of Search and Recognition applications
- Carefully avoid overhead is necessary

Possible future work:

- Algorithm restructuring
- Compiler optimization



WISCONSIN
MADISON

Thank you!

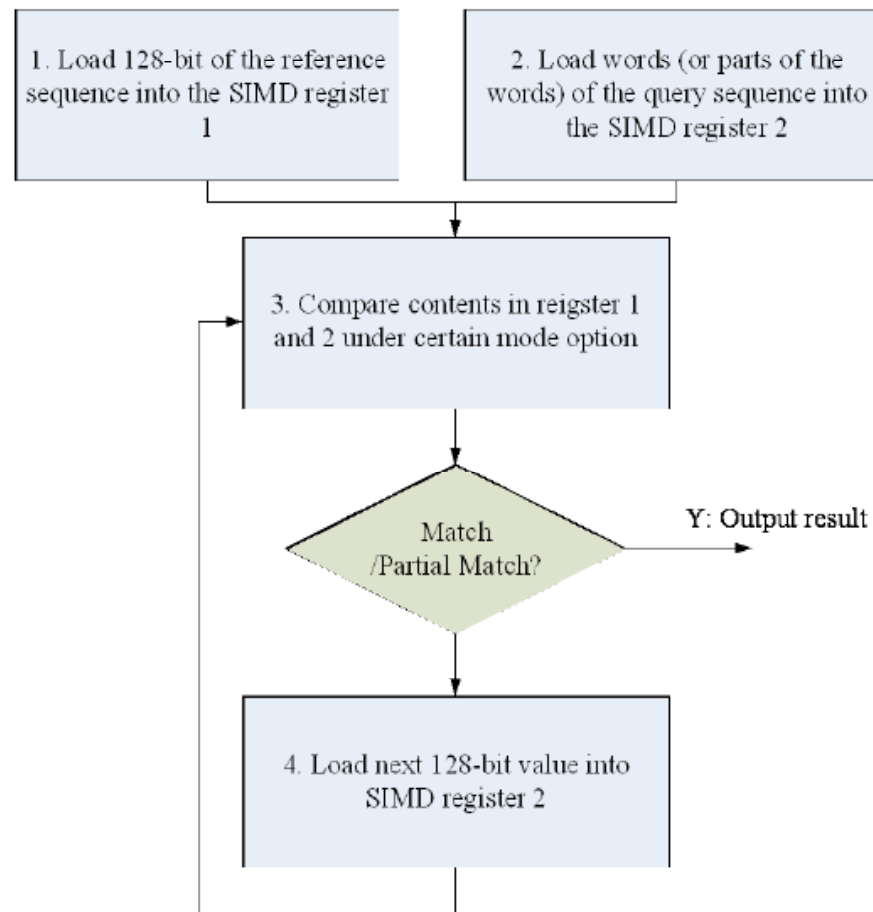


WISCONSIN
MADISON

Extra slides



Optimization with STTNI: General





Code Samples: strcmp

```
int _STTNI_strcmp (const char *p1, const char *p2) {
    const int mode = _SIDD_UBYTE_OPS | _SIDD_CMP_EQUAL_EACH | _SIDD_BIT_MASK
                    | _SIDD_NEGATIVE_POLARITY;
    __m128i smm1 = _mm_loadu_si128 ((__m128i *) p1);
    __m128i smm2 = _mm_loadu_si128 ((__m128i *) p2);
    int ResultIndex;
    while (1) {
        ResultIndex = _mm_cmpistri (smm1, smm2, mode );
        if (ResultIndex != 16) { break; }
        p1 = p1+16;
        p2 = p2+16;
        smm1 = _mm_loadu_si128 ((__m128i *)p1);
        smm2 = _mm_loadu_si128 ((__m128i *)p2);
    }
    p1 = (char *) & smm1;
    p2 = (char *) & smm2;
    if(p1[ResultIndex]<p2[ResultIndex]) return -1;
    if(p1[ResultIndex]>p2[ResultIndex]) return 1;
    return 0;
}
```



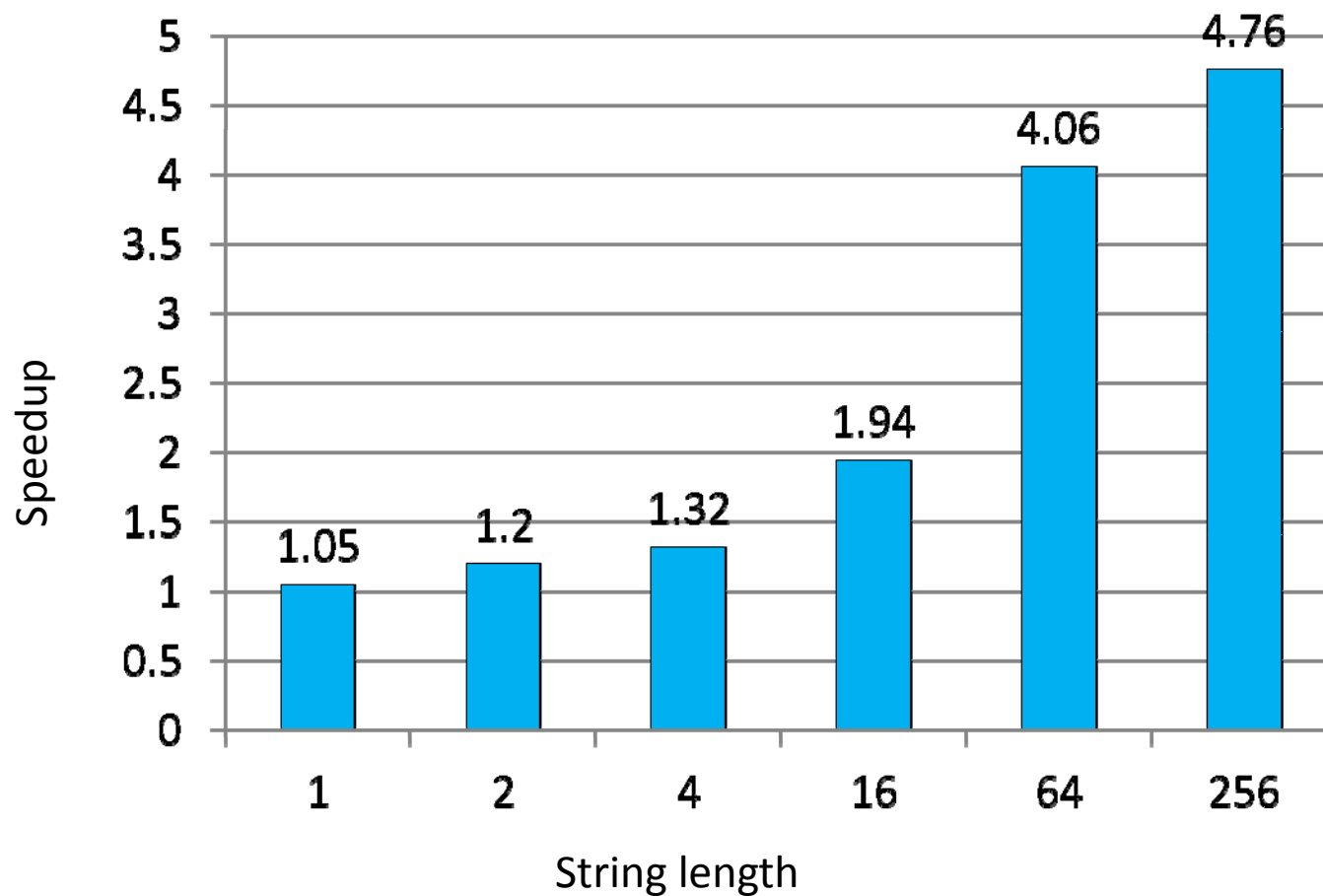
Code Samples: Cache simulator

```
int smm_lookupTags (unsigned long long addr, unsigned int * set) {
    int retval, comparelen;
    __m128i smmTagArray, smmRef;
    unsigned char refchar = CalcTagbits(addr);
    smmRef = _mm_loadu_si128( (__m128i*)&refchar );
    for(unsigned int i=0; i < Assoc; ) {
        smmTagArray = _mm_loadu_si128((__m128i*)(TagMatrix[index(addr)] + i));
        comparelen = 16<(Assoc-i)? 16 : Assoc-i;
        retval = _mm_cmpestri(smmRef, 1, smmTagArray, comparelen, mode);
        if (retval != 16) {
            if(dir[i+retval].lookup(addr) == va_true) {
                *set = i+retval;
                return 1;
            }
            else {      i=i+retval+1;      }
        }
        else {      i=i+16;      }
    }
    return 0;
}
```



Experimental Results

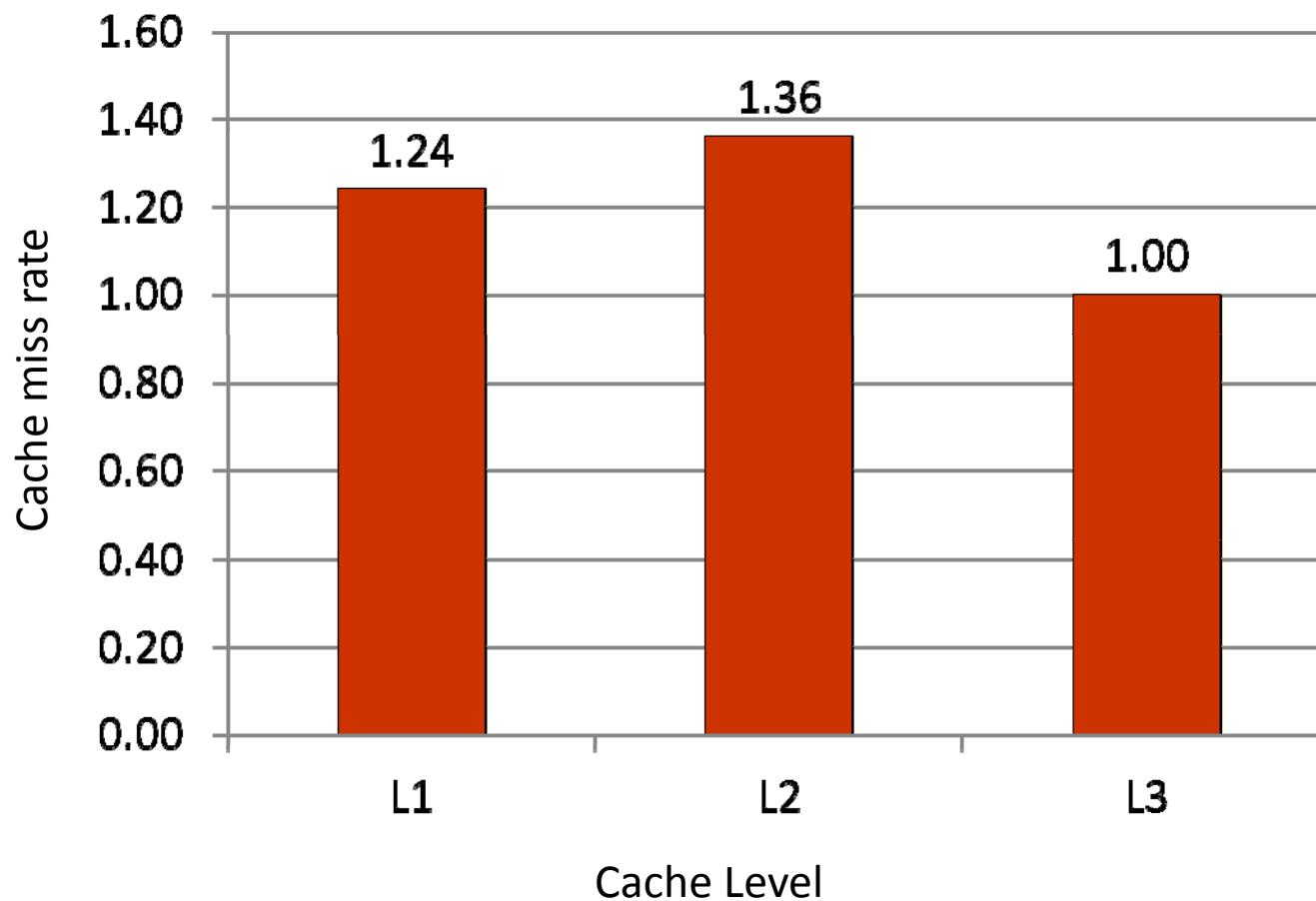
■ String function





Experimental Results

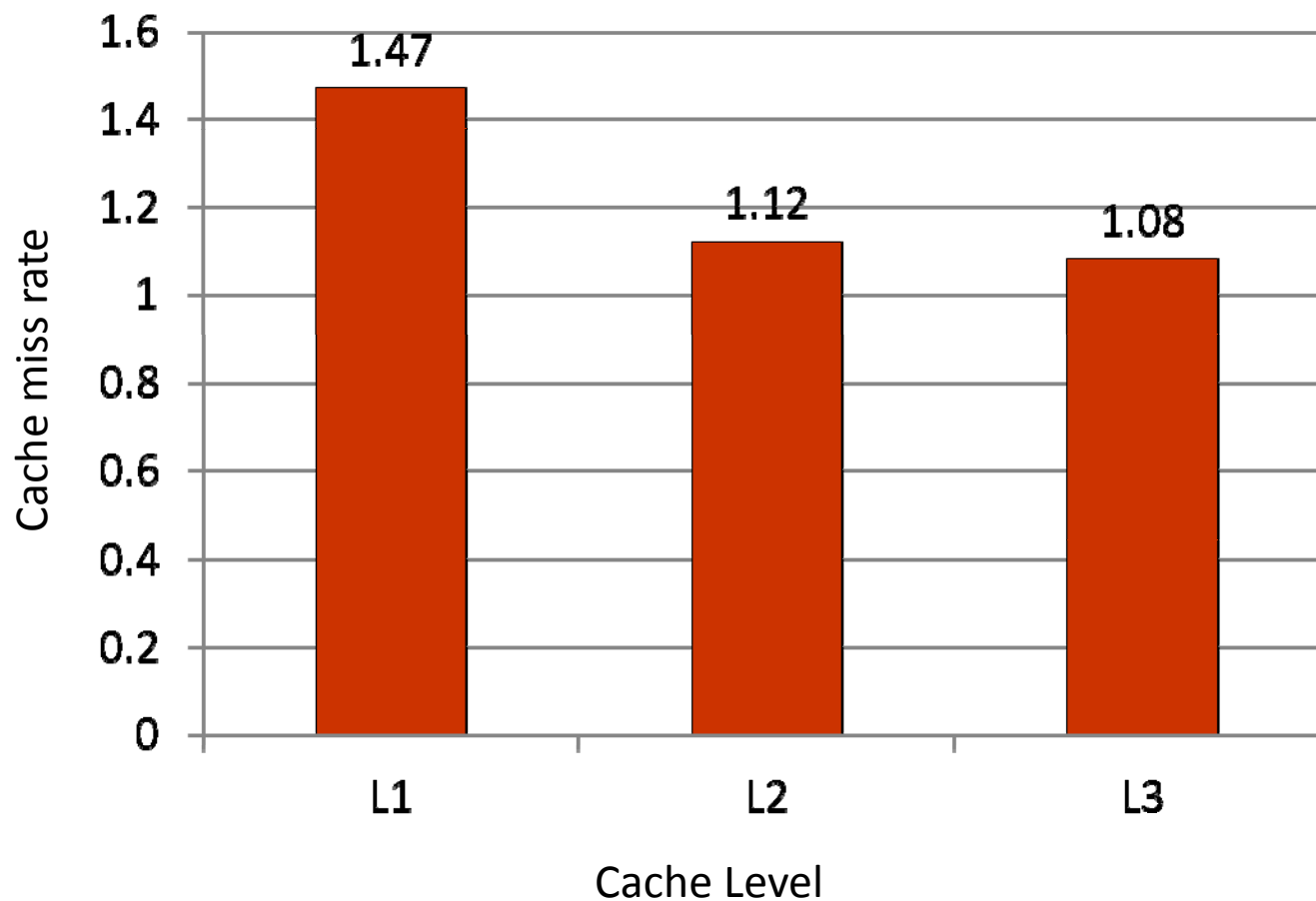
■ Cache Simulator





Experimental Results

■ Template Matching





Experimental Results

■ B+tree algorithm

